

企业级应用软件设计与开发

期末大作业报告

EnterpriseDocAgent

企业知识库 Agentic RAG 智能问答系统

课程名称	企业级应用软件设计与开发
项目名称	EnterpriseDocAgent
方向	方向一：Agentic AI 原生开发
学号	2025302941
姓名	薛佳孟
专业	计算机技术
指导教师	戚欣
提交日期	2026 年 6 月 22 日

代码仓库：<https://github.com/aoao588/cs599-project>

目录

一、选题背景与设计思想	4
1.1 问题定义	4
1.2 现有方案的不足	4
1.3 项目价值	4
1.4 技术路线	4
二、Specs 规格文档 (SDD 核心)	4
2.1 Product Spec —— 用户故事与验收	5
2.2 Architecture Spec —— 模块与状态机	5
2.3 API Spec —— 接口契约	5
三、系统架构与设计	5
3.1 总体分层架构	5
3.2 Agent 交互流程 (LangGraph 状态机)	6
3.3 关键设计决策	7
四、关键实现与代码展示	7
4.1 Agent 核心循环 (状态机编译)	8
4.2 节点定义: grade (结构化输出评估检索质量)	8
4.3 配置与安全 (API Key 环境变量化)	8
4.4 生产级工程实践	8
4.5 AI IDE (Trae CN) 使用	9
五、测试与评估	9
5.1 评估方法	9
5.2 量化结果	9
5.3 对照实验: 朴素 RAG vs Agentic RAG	10
5.4 Demo 展示	11
六、系统升级与扩展	11
6.1 可扩展架构	11
6.2 下一阶段计划	11
6.3 AI 能力演进路径	12
七、课程总结	12
7.1 个人收获	12
7.2 工程思维转变	12
7.3 对课程的建议	12

一、选题背景与设计思想

1.1 问题定义

中型企业内部知识高度碎片化：研发规范、HR 制度、财务流程分散在 Wiki / PDF / 数据库中。员工对请假、报销、规范等问题的高频咨询，占用大量 HR 与资深工程师的时间，新员工上手缓慢。亟需一个能基于企业内部文档自动、准确、可追溯地回答问题的系统。

1.2 现有方案的不足

方案	痛点
关键词全文搜索	只做字面匹配，不理解语义；返回的是文档列表而非直接答案
朴素 RAG（单轮检索+生成）	一次检索定胜负；遇口语化/跨库/需计算的问题易答非所问；无引用难核对；易产生幻觉

1.3 项目价值

本项目构建一个会规划、会反思、带引用、拒绝幻觉的 Agentic RAG 问答系统，把“检索”从一次性动作升级为由 LLM 驱动的多步决策过程。第五章的对照实验证明：在需要定位深层条款的复杂问题上，本系统显著优于朴素 RAG。

1.4 技术路线

- 编排：LangGraph 状态机驱动多步推理（router → rewrite → retrieve → grade → generate → reflect）
- 模型：DeepSeek（LLM）+ 通义 text-embedding-v3（Embedding），全部走 OpenAI 兼容协议
- 存储：Chroma 本地向量库；评估：Ragas（LLM-as-judge）
- 核心技术要素覆盖：SDD 规格驱动、工具使用、状态管理与多步推理、记忆机制、可观测性与评估

二、Specs 规格文档（SDD 核心）

本项目采用规格驱动开发（SDD）：先写规格、再据规格实现。三份规格文档构成工程闭环，完

整版见仓库 docs/specs/。

2.1 Product Spec —— 用户故事与验收

编号	用户故事	关键验收标准
US-01	文档问答带可追溯来源	答案每个事实点带 [n] 角标 + 来源文件
US-02	跨库提问自动联检	报销+HR 类问题同时检索多个知识域
US-03	检索不足自动重检	首轮不足时改写 Query 重试，最多 2 轮
US-05	拒绝幻觉	知识库无依据时明确回答“未找到”，不编造

2.2 Architecture Spec —— 模块与状态机

定义四层架构、AgentState 状态结构、6 节点状态机与条件边、以及关键设计决策（见第三章）。

2.3 API Spec —— 接口契约

定义各模块 Python 接口契约（LLMClient / EmbeddingClient / vectorstore / 各节点签名），以及面向未来的 HTTP /api/v1/ask 与 MCP Server（vector_search / sql_query）接口，体现可扩展性。

三、系统架构与设计

3.1 总体分层架构

系统分为四层，层间解耦：接入层（CLI / Web）负责输入输出；Agent 编排层用 LangGraph 状态机驱动多步推理；服务层封装 LLM、Embedding、向量库；外部 API 全部走 OpenAI 兼容协议，更换厂商只需修改 .env。

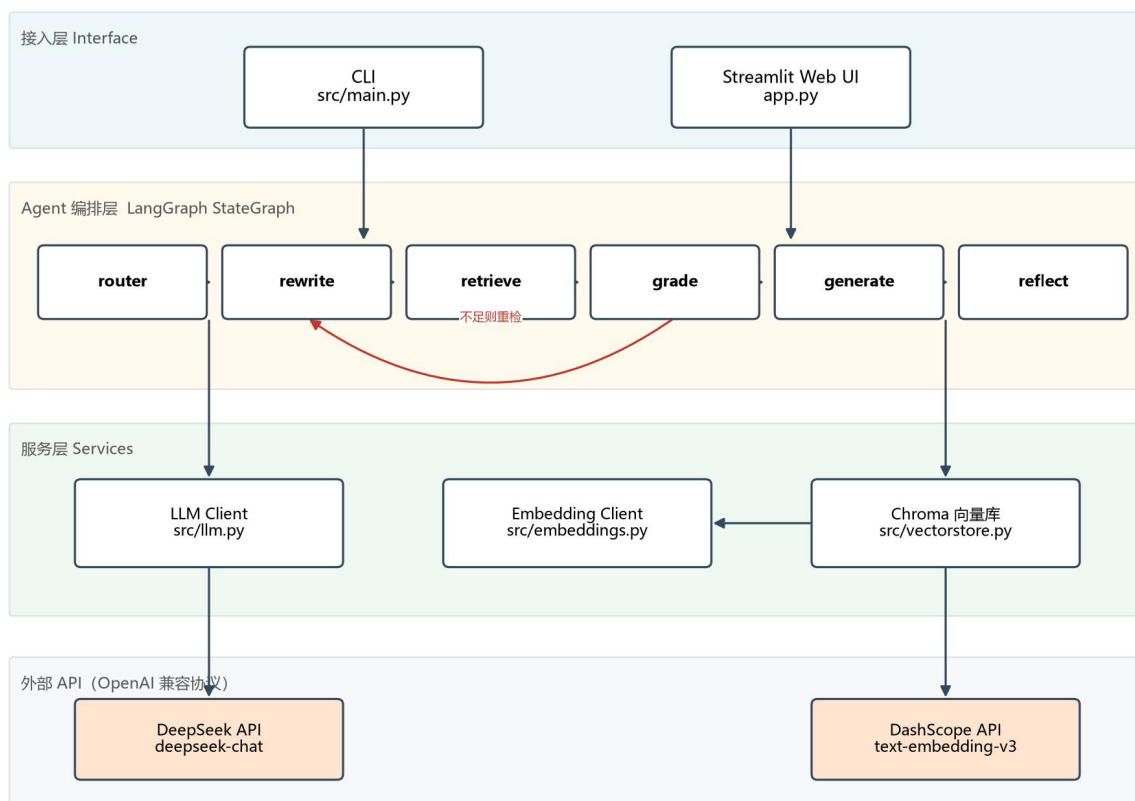


图 3-1 总体分层架构

3.2 Agent 交互流程 (LangGraph 状态机)

核心是一个带反馈环的状态机：`router` 判定意图后，经 `rewrite` 多查询改写、`retrieve` 检索、`grade` 评估；检索不足则回到 `rewrite` 重检，充分则 `generate` 带引用作答，再由 `reflect` 反思校验。循环由 `rewrite_count` 预算（最多 3 次）与 `recursion_limit=15` 双重兜底，knowledge 库外问题在 `grade=irrelevant` 时早退，避免空转。

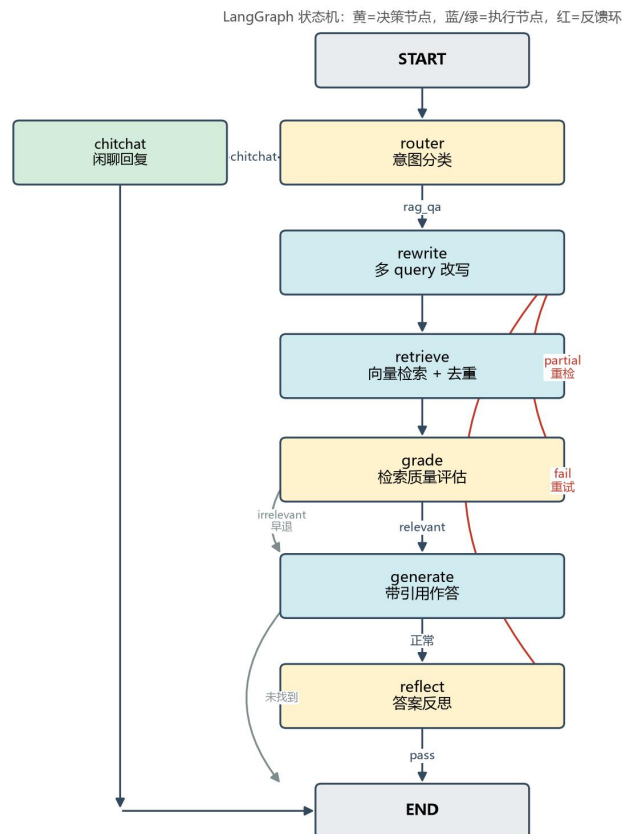


图 3-2 LangGraph 状态机（黄=决策，蓝/绿=执行，红=反馈环）

3.3 关键设计决策

决策点	选择	理由
编排框架	LangGraph	显式状态 + 条件边 + 反思循环，天然契合 Agentic RAG
Embedding	通义 text-embedding-v3	DeepSeek 无 embedding API；通义中文强、成本低
向量库	Chroma	零运维、本地持久化、SQLite 后端
结构化输出	DeepSeek JSON mode	grade/reflect 节点稳定解析判定结果

四、关键实现与代码展示

4.1 Agent 核心循环（状态机编译）

在 `src/graph/build.py` 中用 `LangGraph` 的 `StateGraph` 声明节点与条件边，编译为可执行图：

```
g = StateGraph(AgentState)
g.add_node("router", node_router)
g.add_node("rewrite", node_rewrite)
g.add_node("retrieve", node_retrieve)
g.add_node("grade", node_grade)
g.add_node("generate", node_generate)
g.add_node("reflect", node_reflect)

g.add_edge(START, "router")
g.add_conditional_edges("grade", _route_after_grade,
    {"generate": "generate", "rewrite": "rewrite"})
g.add_conditional_edges("reflect", _route_after_reflect,
    {"rewrite": "rewrite", END: END})
return g.compile()
```

4.2 节点定义：grade（结构化输出评估检索质量）

grade 节点用 `DeepSeek` 的 `JSON mode` 输出结构化判定，是“质量控制”的关键：

```
def node_grade(state: AgentState) -> dict:
    out = get_llm().chat_json([
        {"role": "system", "content": _GRADE_SYS},
        {"role": "user", "content":
            f"问题: {state['question']}\n 检索片段: \n{...}"},
    ])
    grade = out.get("grade", "partial") # relevant|partial|irrelevant
    return {"grade": grade, "trace": _append_trace(state, ...)}
```

4.3 配置与安全（API Key 环境变量化）

`src/config.py` 用 `pydantic-settings` 从 `.env` 读取配置，API Key 全程以环境变量注入，绝不硬编码，符合学术纪律与生产安全要求：

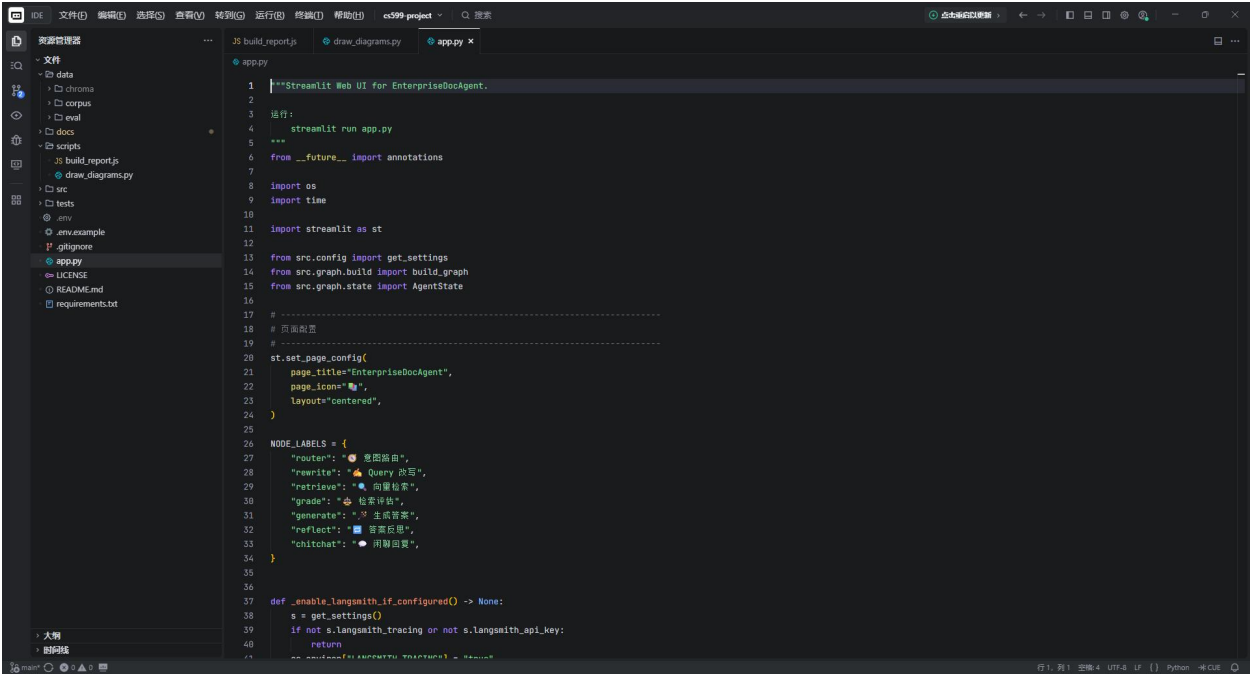
```
class Settings(BaseSettings):
    deepseek_api_key: SecretStr # 从 .env 注入，不硬编码
    dashscope_api_key: SecretStr
    deepseek_model: str = "deepseek-chat"
    retrieve_top_k: int = 5
    max_reflect_rounds: int = 2
    request_timeout: int = 60
    model_config = SettingsConfigDict(env_file=".env")
```

4.4 生产级工程实践

- API 超时控制（`timeout=60s`）+ `tenacity` 指数退避重试，避免单次调用挂起死等

- JSON 解析失败自动追加“严格输出 JSON”系统消息并重试一次
- 流式执行 (graph.stream) 打印每个节点耗时，可观测、可定位慢节点
- recursion_limit 硬兜底，杜绝任何残留循环烧 token

4.5 AI IDE (Trae CN) 使用



五、测试与评估

5.1 评估方法

知识库 25 篇企业文档 / 44 个向量片段（含 21 篇干扰文档，检验抗干扰检索）；Golden QA 共 10 条（8 条知识库内 + 2 条知识库外 OOD）；评估工具 Ragas（LLM-as-judge，评判用 DeepSeek，相似度用通义 Embedding）。

5.2 量化结果

指标	得分	目标	达标
faithfulness（忠实度/无幻觉）	0.824	≥ 0.85	接近
answer_relevancy（答案相关性）	0.887	≥ 0.85	达标
context_precision（检索精度）	0.938	≥ 0.90	达标

指标	得分	目标	达标
OOD 拒答率	100%	100%	达标

faithfulness 的扣分主要来自：当 Agent 为说明计算规则而自行举例推算时，这些具体数字不在原文，会被判为部分不忠实——反映了“推理能力 vs 严格忠实”的固有张力，也佐证了 Ragas 的判别力。

5.3 对照实验：朴素 RAG vs Agentic RAG

为验证 Agentic 架构本身的价值，实现朴素 RAG baseline（单轮 retrieve→generate，无 rewrite/grade/reflect）做对照。

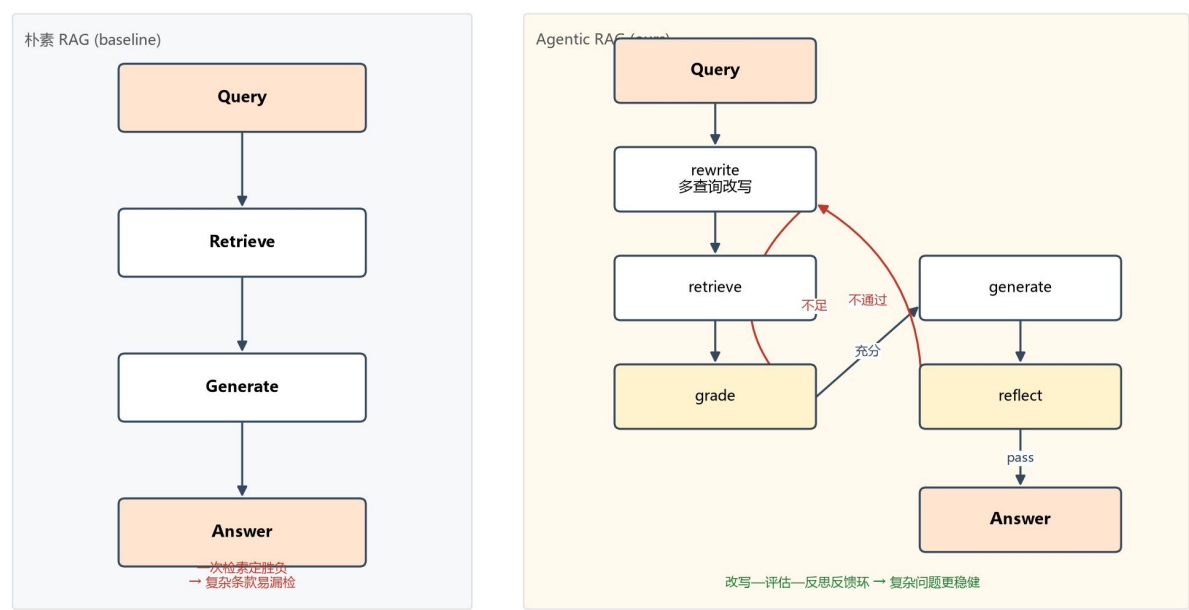


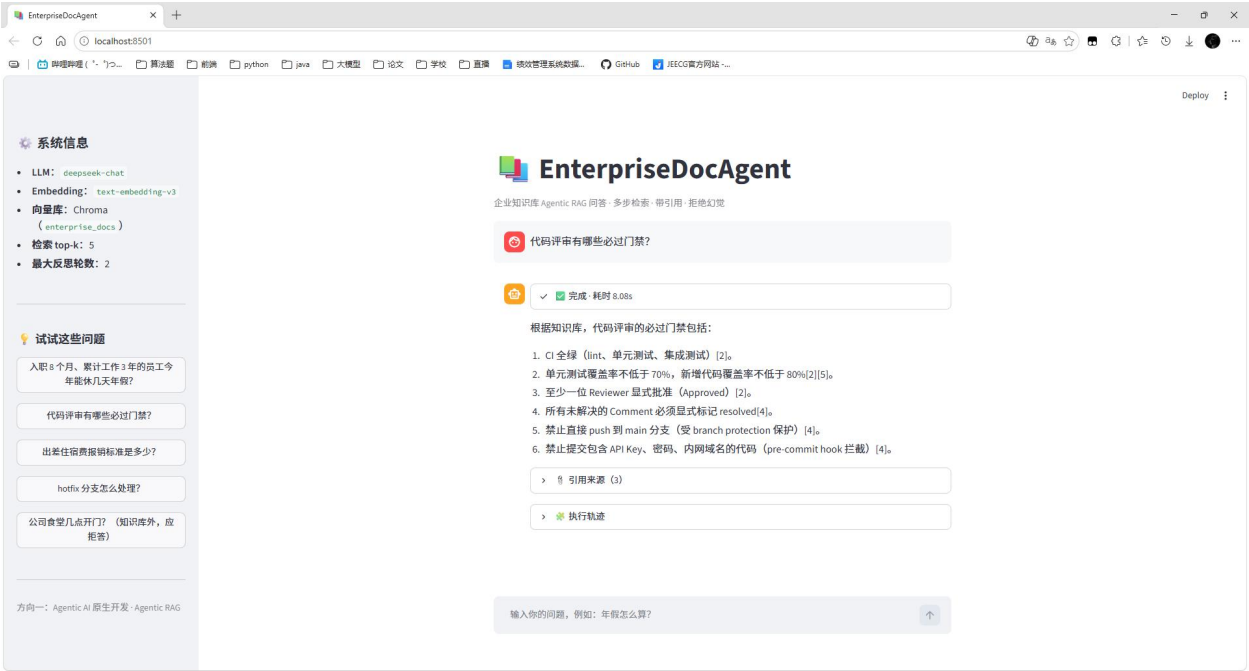
图 5-1 朴素 RAG（直线）vs Agentic RAG（反馈环）

样例	朴素 RAG	Agentic RAG
qa01 年假天数（直接查表）	满分	满分（打平）
qa02 首年折算（需定位深层条款）	0 / 0 / 0 漏检失败	正确作答
OOD 拒答率	100%	100%

关键案例 qa02：知识库中确实包含“首年折算公式”，但朴素 RAG 单轮检索命中开头大片段、

公式被上下文淹没，竟回答“未找到”（假阴性）；Agentic RAG 凭 rewrite 生成“首年折算”等子查询直接命中条款、grade 评估通过后作答。结论：Agentic 架构的增益不在简单问题、而在“需主动改写检索、跨片段定位”的复杂问题上——这正是企业知识库的常态。

5.4 Demo 展示



六、系统升级与扩展

6.1 可扩展架构

全链路 OpenAI 兼容协议，更换 LLM/Embedding 厂商只需改 .env，零代码改动；节点之间松耦合，可独立增删与替换。

6.2 下一阶段计划

- MCP 化：将 vector_search / sql_query 抽为独立 MCP Server，跨项目复用，融合前沿协议
- 多智能体：将 grade 拆为独立 Critic Agent，与 Retriever Agent 以 supervisor 模式编排
- 检索增强：引入 BGE-reranker 重排、MarkdownHeader 感知切分，提升命中率
- 结构化数据：接入 DuckDB，支持自然语言查表（sql_query 工具）

6.3 AI 能力演进路径

单 Agent → 多 Agent 协作 → 跨会话长期记忆 → 云端部署 + 流式输出 (SSE)。

七、课程总结

7.1 个人收获

本项目让我完成了从“代码编写者”到“智能体编排者”的转变。最大的体会是：Agentic 系统的难点不在单个 LLM 调用，而在状态流转与边界处理。例如开发中遇到 grade 节点反复判 partial 导致的死循环、知识库外问题的过度重试，都是通过引入统一的重试预算 (rewrite_count) 与早退逻辑才解决——这类“编排级”的工程问题是传统 CRUD 开发中不会遇到的。

7.2 工程思维转变

SDD（先写规格再实现）让我先想清楚“系统该有什么行为”再动手，减少了返工。可观测性同样关键：给状态机加上节点级耗时打印后，才一眼定位出“评估慢是 LLM-as-judge 的固有成本而非 bug”。此外，处理 ragas 与 langchain 版本不兼容、在有限 API 额度下做成本权衡，都让我体会到工程落地中“约束驱动设计”的真实含义。

7.3 对课程的建议

降低 API 成本门槛，协调一定的免费模型额度。